1. Write a program to take a list of 10 farmers, fruits they sell, their cities & the quantity of fruits they have as input. Save the data to notepad before & after sorting based on quantity in tabular manner. Also perform search operation based on farmers' name.

Program:

```
import java.io.*;
import java.util.*;

class Farmer {
    String name, fruit, city;
    int quantity;

    Farmer(String name, String fruit, String city, int quantity) {
        this.name = name;
        this.fruit = fruit;
        this.city = city;
        this.quantity = quantity;
    }
}

public class Main {

    // Display in tabular format
    static void display(List<Farmer> list) {
        System.out.println("------------------------------------------------------------");
        System.out.printf("%-15s %-15s %-15s %-10s\n", "Name", "Fruit", "City", "Quantity");
        System.out.println("------------------------------------------------------------");

        for (Farmer f : list) {
            System.out.printf("%-15s %-15s %-15s %-10d\n",
                    f.name, f.fruit, f.city, f.quantity);
        }
    }

    // Save data to file
    static void saveToFile(List<Farmer> list, String filename) {
        try {
            BufferedWriter bw = new BufferedWriter(new FileWriter(filename));

            bw.write("------------------------------------------------------------\n");
            bw.write(String.format("%-15s %-15s %-15s %-10s\n",
                    "Name", "Fruit", "City", "Quantity"));
            bw.write("------------------------------------------------------------\n");
```

```
        for (Farmer f : list) {
            bw.write(String.format("%-15s %-15s %-15s %-10d\n",
                    f.name, f.fruit, f.city, f.quantity));
        }

        bw.close();
        System.out.println("Saved in file: " + filename);
    } catch (IOException e) {
        System.out.println("File error!");
    }
}

// Search farmer by name
static void search(List<Farmer> list, String key) {
    boolean found = false;

    for (Farmer f : list) {
        if (f.name.equalsIgnoreCase(key)) {
            System.out.println("\nFarmer Found:");
            System.out.println("Name: " + f.name);
            System.out.println("Fruit: " + f.fruit);
            System.out.println("City: " + f.city);
            System.out.println("Quantity: " + f.quantity);
            found = true;
            break;
        }
    }

    if (!found) {
        System.out.println("Farmer not found!");
    }
}

public static void main(String[] args) {
    Scanner sc = new Scanner(System.in);
    List<Farmer> list = new ArrayList<>();

    // Input 10 farmers
    for (int i = 1; i <= 10; i++) {
        System.out.println("\nEnter Farmer " + i + " details:");

        System.out.print("Name: ");
        String name = sc.nextLine();

        System.out.print("Fruit: ");
        String fruit = sc.nextLine();

        System.out.print("City: ");
```

```
            String city = sc.nextLine();

            System.out.print("Quantity: ");
            int quantity = sc.nextInt();
            sc.nextLine(); // fix buffer

            list.add(new Farmer(name, fruit, city, quantity));
        }

        // Before sorting
        System.out.println("\n--- BEFORE SORTING ---");
        display(list);
        saveToFile(list, "before_sort.txt");

        // Sorting by quantity
        Collections.sort(list, (a, b) -> a.quantity - b.quantity);

        // After sorting
        System.out.println("\n--- AFTER SORTING ---");
        display(list);
        saveToFile(list, "after_sort.txt");

        // Search
        System.out.print("\nEnter farmer name to search: ");
        String key = sc.nextLine();
        search(list, key);

        sc.close();
    }
}
```

Output:

```
Enter Farmer 1 details:
Name: Arjun
Fruit: Pineapple
City: Goa
Quantity: 45

Enter Farmer 2 details:
Name: Meena
Fruit: Papaya
City: chennai
Quantity: 25

Enter Farmer 3 details:
Name: Vikram
Fruit: guava
```

```
City: lucknow
Quantity: 65

Enter Farmer 4 details:
Name: Sunita
Fruit: Watermelon
City: Patna
Quantity: 35

Enter Farmer 5 details:
Name: Deepak
Fruit: lichi
City: pune
Quantity: 75

Enter Farmer 6 details:
Name: Kavita
Fruit: Apple
City: Simla
Quantity: 35

Enter Farmer 7 details:
Name: Rohit
Fruit: Mango
City: Hydrabad
Quantity: 6 55

Enter Farmer 8 details:
Name: Anjali
Fruit: Banana
City: Kolkata
Quantity: 30

Enter Farmer 9 details:
Name: Suresh
Fruit: Orange
City: Jaipur
Quantity: 40

Enter Farmer 10 details:
Name: Pankaj
Fruit: grapes
City: Nasik
Quantity: 50

--- BEFORE SORTING ---
------------------------------------------------------------
Name        Fruit        City        Quantity
```

```
------------------------------------------------------------
Arjun        Pineapple      Goa          45
Meena        Papaya         chennai      25
Vikram       guava          lucknow      65
Sunita       Watermelon     Patna        35
Deepak       lichi          pune         75
Kavita       Apple          Simla        35
Rohit        Mango          Hydrabad     55
Anjali       Banana         Kolkata      30
Suresh       Orange         Jaipur       40
Pankaj       grapes         Nasik        50
Saved in file: before_sort.txt

--- AFTER SORTING ---
------------------------------------------------------------
Name         Fruit          City         Quantity
------------------------------------------------------------
Meena        Papaya         chennai      25
Anjali       Banana         Kolkata      30
Sunita       Watermelon     Patna        35
Kavita       Apple          Simla        35
Suresh       Orange         Jaipur       40
Arjun        Pineapple      Goa          45
Pankaj       grapes         Nasik        50
Rohit        Mango          Hydrabad     55
Vikram       guava          lucknow      65
Deepak       lichi          pune         75
Saved in file: after_sort.txt

Enter farmer name to search: Kavita

Farmer Found:
Name: Kavita
Fruit: Apple
City: Simla
Quantity: 35
```

2. Write a program to take quantity & quality of fruits Mango, Apple & Papaya as input. Quality may be good, bad or average. If entered quantity of fruit is greater than 1000, store it as half. Perform search for a given fruit with given quality.

Program:

```
import java.util.*;
```

```
class Fruit {
   String name;
   String quality;
   int quantity;

   Fruit(String name, String quality, int quantity) {
      this.name = name;
      this.quality = quality;

      // Condition: if quantity > 1000, store half
      if (quantity > 1000) {
         this.quantity = quantity / 2;
      } else {
         this.quantity = quantity;
      }
   }
}

public class Main {

   // Search function
   static void search(Fruit[] fruits, String name, String quality) {
      boolean found = false;

      for (Fruit f : fruits) {
         if (f.name.equalsIgnoreCase(name) &&
            f.quality.equalsIgnoreCase(quality)) {

            System.out.println("\nFruit Found:");
            System.out.println("Name: " + f.name);
            System.out.println("Quality: " + f.quality);
            System.out.println("Quantity: " + f.quantity);
            found = true;
         }
      }

      if (!found) {
         System.out.println("\nFruit not found!");
      }
   }

   public static void main(String[] args) {
      Scanner sc = new Scanner(System.in);
      Fruit[] fruits = new Fruit[3];

      String[] names = {"Mango", "Apple", "Papaya"};

      // Input
```

```
        for (int i = 0; i < 3; i++) {
            System.out.println("\nEnter details for " + names[i]);

            System.out.print("Quantity: ");
            int qty = sc.nextInt();
            sc.nextLine();

            System.out.print("Quality (good/average/bad): ");
            String quality = sc.nextLine();

            fruits[i] = new Fruit(names[i], quality, qty);
        }

        // Display stored data
        System.out.println("\n--- Stored Data ---");
        System.out.println("----------------------------------");
        System.out.printf("%-10s %-10s %-10s\n", "Fruit", "Quality", "Quantity");
        System.out.println("----------------------------------");

        for (Fruit f : fruits) {
            System.out.printf("%-10s %-10s %-10d\n",
                    f.name, f.quality, f.quantity);
        }

        // Search
        System.out.print("\nEnter fruit name to search: ");
        String name = sc.nextLine();

        System.out.print("Enter quality: ");
        String quality = sc.nextLine();

        search(fruits, name, quality);

        sc.close();
    }
}
```

Output:

```
Enter details for Mango
Quantity: 1300
Quality (good/average/bad): good

Enter details for Apple
Quantity: 900
Quality (good/average/bad): average
```

```
Enter details for Papaya
Quantity: 1500.1500150151
Quality (good/average/bad): bad

--- Stored Data ---
----------------------------------
Fruit      Quality    Quantity
----------------------------------
Mango      good       650
Apple      average    900
Papaya     bad        750

Enter fruit name to search: Mango
Enter quality: good

Fruit Found:
Name: Mango
Quality: good
Quantity: 650
```

3. There are four orchards, namely A, B, C & D with fruits in them as Apple, Litchi, Mango, Apple respectively. The qualities of the fruits in the orchards are good, good, good & bad respectively.
The available orchards in a tree-view.
Search for a given fruit in the available orchards after taking input from the user & print the tree.

Program:

```
import java.util.*;

// Tree Node
class Orchard {
    String name;
    String fruit;
    String quality;

    Orchard(String name, String fruit, String quality) {
        this.name = name;
        this.fruit = fruit;
        this.quality = quality;
    }
}
```

```
public class Main {

    // Function to print tree
    static void printTree(Orchard[] orchards) {
        System.out.println("\n--- Orchard Tree ---");
        System.out.println("Orchards");

        for (Orchard o : orchards) {
            System.out.println(" ├── " + o.name);
            System.out.println(" │   ├── Fruit: " + o.fruit);
            System.out.println(" │   └── Quality: " + o.quality);
        }
    }

    // Search function
    static void search(Orchard[] orchards, String fruit) {
        boolean found = false;

        for (Orchard o : orchards) {
            if (o.fruit.equalsIgnoreCase(fruit)) {
                System.out.println("\nFruit found in Orchard: " + o.name);
                System.out.println("Quality: " + o.quality);
                found = true;
            }
        }

        if (!found) {
            System.out.println("\nFruit not found in any orchard!");
        }
    }

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        // Given data
        Orchard[] orchards = {
            new Orchard("A", "Apple", "good"),
            new Orchard("B", "Litchi", "good"),
            new Orchard("C", "Mango", "good"),
            new Orchard("D", "Apple", "bad")
        };

        // Print tree
        printTree(orchards);

        // Search
        System.out.print("\nEnter fruit to search: ");
```

```
        String fruit = sc.nextLine();

        search(orchards, fruit);

        sc.close();
    }
}
```

Output:

```
--- Orchard Tree ---
Orchards
 ├── A
 │    ├── Fruit: Apple
 │    └─► Quality: good
 ├── B
 │    ├── Fruit: Litchi
 │    └── Quality: good
 ├── C
 │    ├── Fruit: Mango
 │    └── Quality: good
 ├── D
 │    ├── Fruit: Apple
 │    └── Quality: bad
 │

Enter fruit to search: Apple

Fruit found in Orchard: A
Quality: good

Fruit found in Orchard: D
Quality: bad
```

4. Take a map provided as a graph & display all the subtrees of connected points.

Program:

```
import java.util.*;

public class Main {

    // DFS function
    static void dfs(int node, boolean[] visited, ArrayList<ArrayList<Integer>> graph) {
```

```
        visited[node] = true;
        System.out.print(node + " ");

        for (int neighbor : graph.get(node)) {
            if (!visited[neighbor]) {
                dfs(neighbor, visited, graph);
            }
        }
    }

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        System.out.print("Enter number of vertices: ");
        int v = sc.nextInt();

        System.out.print("Enter number of edges: ");
        int e = sc.nextInt();

        // Create graph
        ArrayList<ArrayList<Integer>> graph = new ArrayList<>();

        for (int i = 0; i < v; i++) {
            graph.add(new ArrayList<>());
        }

        System.out.println("Enter edges (u v): ");
        for (int i = 0; i < e; i++) {
            int u = sc.nextInt();
            int vtx = sc.nextInt();

            graph.get(u).add(vtx);
            graph.get(vtx).add(u); // undirected graph
        }

        boolean[] visited = new boolean[v];

        System.out.println("\nSubtrees (Connected Components):");

        // Find connected components
        for (int i = 0; i < v; i++) {
            if (!visited[i]) {
                System.out.print("Component: ");
                dfs(i, visited, graph);
                System.out.println();
            }
        }
```

```
        sc.close();
    }
}
```

Output:

```
Enter number of vertices: 6
Enter number of edges: 4
Enter edges (u v):
0 1
1 2
2 3
3 4

Subtrees (Connected Components):
Component: 0 1 2 3 4
Component: 5
```

5. Enter entry point & goal point & produce the matching subtree of the graph.

Program:

```
import java.util.*;

public class Main {

    // DFS to find path
    static boolean dfs(int current, int goal, boolean[] visited,
                       ArrayList<ArrayList<Integer>> graph,
                       List<Integer> path) {

        visited[current] = true;
        path.add(current);

        if (current == goal) {
            return true;
        }

        for (int neighbor : graph.get(current)) {
            if (!visited[neighbor]) {
                if (dfs(neighbor, goal, visited, graph, path)) {
                    return true;
                }
            }
        }
```

```
    // backtrack if not found
    path.remove(path.size() - 1);
    return false;
}

public static void main(String[] args) {
    Scanner sc = new Scanner(System.in);

    System.out.print("Enter number of vertices: ");
    int v = sc.nextInt();

    System.out.print("Enter number of edges: ");
    int e = sc.nextInt();

    ArrayList<ArrayList<Integer>> graph = new ArrayList<>();

    for (int i = 0; i < v; i++) {
        graph.add(new ArrayList<>());
    }

    System.out.println("Enter edges (u v): ");
    for (int i = 0; i < e; i++) {
        int u = sc.nextInt();
        int w = sc.nextInt();

        graph.get(u).add(w);
        graph.get(w).add(u); // undirected
    }

    System.out.print("Enter entry point: ");
    int start = sc.nextInt();

    System.out.print("Enter goal point: ");
    int goal = sc.nextInt();

    boolean[] visited = new boolean[v];
    List<Integer> path = new ArrayList<>();

    boolean found = dfs(start, goal, visited, graph, path);

    if (found) {
        System.out.println("\nMatching Subtree (Path):");
        for (int node : path) {
            System.out.print(node + " ");
        }
    } else {
        System.out.println("\nNo path found!");
    }
```

```
        sc.close();
    }
}
```

Output:

```
Enter number of vertices: 6
Enter number of edges: 5
Enter edges (u v):
0 1
1 2
2 3
3 4
454 4     5
Enter entry point: 0
Enter goal point: 4

Matching Subtree (Path):
0 1 2 3 4
```

6. Give the weightage for each edge of nodes, and produce the shortest path from start to end node.

Program:

```
import java.util.*;

class Pair {
    int node, distance;

    Pair(int node, int distance) {
        this.node = node;
        this.distance = distance;
    }
}

public class Main {

    public static void dijkstra(ArrayList<ArrayList<Pair>> graph, int V, int start, int end) {

        int[] dist = new int[V];
        int[] parent = new int[V];

        Arrays.fill(dist, Integer.MAX_VALUE);
```

```
        Arrays.fill(parent, -1);

        PriorityQueue<Pair> pq = new PriorityQueue<>(Comparator.comparingInt(x ->
x.distance));

        dist[start] = 0;
        pq.add(new Pair(start, 0));

        while (!pq.isEmpty()) {
            Pair current = pq.poll();
            int u = current.node;

            for (Pair neighbor : graph.get(u)) {
                int v = neighbor.node;
                int weight = neighbor.distance;

                if (dist[u] + weight < dist[v]) {
                    dist[v] = dist[u] + weight;
                    parent[v] = u;
                    pq.add(new Pair(v, dist[v]));
                }
            }
        }

        // Print shortest distance
        System.out.println("\nShortest Distance: " + dist[end]);

        // Print path
        System.out.print("Shortest Path: ");
        printPath(end, parent);
    }

    // Function to print path
    static void printPath(int node, int[] parent) {
        if (node == -1) return;
        printPath(parent[node], parent);
        System.out.print(node + " ");
    }

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        System.out.print("Enter number of vertices: ");
        int V = sc.nextInt();

        System.out.print("Enter number of edges: ");
        int E = sc.nextInt();
```

```
        ArrayList<ArrayList<Pair>> graph = new ArrayList<>();

        for (int i = 0; i < V; i++) {
            graph.add(new ArrayList<>());
        }

        System.out.println("Enter edges (u v weight):");
        for (int i = 0; i < E; i++) {
            int u = sc.nextInt();
            int v = sc.nextInt();
            int w = sc.nextInt();

            graph.get(u).add(new Pair(v, w));
            graph.get(v).add(new Pair(u, w)); // undirected graph
        }

        System.out.print("Enter start node: ");
        int start = sc.nextInt();

        System.out.print("Enter end node: ");
        int end = sc.nextInt();

        dijkstra(graph, V, start, end);

        sc.close();
    }
}
```

Output:

```
Enter number of vertices: 5
Enter number of edges: 6
Enter edges (u v weight):
0 1 20 11 0 110 10 0                    1 2
02 42 2       0 2 3
1 2 1
131   7    3    3 7
2 4 3
343   4 1
Enter start node: 0
Enter end node: 4

Shortest Distance: 6
Shortest Path: 0 2 4
```

7.Write a program to demonstrate the use of all keywords made for exception handling.

Program:

```
import java.util.*;

public class Main {

    // Method using 'throws'
    static void checkNumber(int num) throws Exception {
        if (num < 0) {
            // using 'throw'
            throw new Exception("Number cannot be negative!");
        } else {
            System.out.println("Valid number: " + num);
        }
    }

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        try {
            // try block
            System.out.print("Enter a number: ");
            int num = sc.nextInt();

            // calling method that may throw exception
            checkNumber(num);

            // Arithmetic Exception example
            System.out.print("Enter divisor: ");
            int d = sc.nextInt();

            int result = 100 / d;
            System.out.println("Result: " + result);

        } catch (ArithmeticException e) {
            // catch specific exception
            System.out.println("Error: Cannot divide by zero!");

        } catch (Exception e) {
            // catch general exception
            System.out.println("Exception: " + e.getMessage());

        } finally {
            // always executes
            System.out.println("Program execution completed.");
```

```
        }

        sc.close();
    }
}
```

Output:

```
Enter a number: 10
Valid number: 10
Enter divisor: 2
Result: 50
Program execution completed.
```

8. Write a program to withraw money from a bank account, if the withdrawn money is greater than the balance, then it throws NoMoneyException.

Program:

```
import java.util.*;

// Custom Exception
class NoMoneyException extends Exception {
    NoMoneyException(String message) {
        super(message);
    }
}

class BankAccount {
    int balance;

    BankAccount(int balance) {
        this.balance = balance;
    }

    // Withdraw method using throws
    void withdraw(int amount) throws NoMoneyException {
        if (amount > balance) {
            // throw custom exception
            throw new NoMoneyException("Insufficient Balance!");
        } else {
            balance = balance - amount;
            System.out.println("Withdrawal Successful!");
            System.out.println("Remaining Balance: " + balance);
```

```
        }
    }
}

public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        System.out.print("Enter initial balance: ");
        int balance = sc.nextInt();

        BankAccount acc = new BankAccount(balance);

        try {
            System.out.print("Enter amount to withdraw: ");
            int amount = sc.nextInt();

            acc.withdraw(amount);

        } catch (NoMoneyException e) {
            System.out.println("Exception: " + e.getMessage());
        }

        sc.close();
    }
}
```

Output:

```
Enter initial balance: 5000
Enter amount to withdraw: 2000
Withdrawal Successful!
Remaining Balance: 3000
```

9. Write a program which enables to use a platform by a train one at a time.

Program:

```
import java.util.concurrent.Semaphore;

// Shared resource: Platform
class Platform {
    Semaphore sem = new Semaphore(1); // only 1 train allowed
```

```
    void usePlatform(String trainName) {
        try {
            System.out.println(trainName + " is waiting for platform...");

            sem.acquire(); // request access

            System.out.println(trainName + " is using the platform...");
            Thread.sleep(2000); // simulate time

            System.out.println(trainName + " has left the platform.");

            sem.release(); // release access

        } catch (InterruptedException e) {
            System.out.println(e);
        }
    }
}

// Train thread
class Train extends Thread {
    Platform p;
    String name;

    Train(Platform p, String name) {
        this.p = p;
        this.name = name;
    }

    public void run() {
        p.usePlatform(name);
    }
}

public class Main {
    public static void main(String[] args) {

        Platform platform = new Platform();

        Train t1 = new Train(platform, "Train A");
        Train t2 = new Train(platform, "Train B");
        Train t3 = new Train(platform, "Train C");

        t1.start();
        t2.start();
        t3.start();
    }
```

```
}
```

Output:

```
Train A is waiting for platform...
Train A is using the platform...
Train B is waiting for platform...
Train C is waiting for platform...
Train A has left the platform.
Train B is using the platform...
Train B has left the platform.
Train C is using the platform...
Train C has left the platform.
```

10.Write a programme of Restaurant and Waiter, which accept order if kitchen if free and sheff deliver orders if he is free.

Program:

```
class Kitchen {
    private boolean orderReady = false;
    private String order;

    // Waiter places order
    synchronized void placeOrder(String order) {
        try {
            while (orderReady) {
                wait(); // wait if kitchen busy
            }

            this.order = order;
            System.out.println("Waiter placed order: " + order);

            orderReady = true;
            notify(); // notify chef

        } catch (InterruptedException e) {
            System.out.println(e);
        }
    }

    // Chef prepares and delivers
    synchronized void prepareOrder() {
```

```
        try {
            while (!orderReady) {
                wait(); // wait if no order
            }

            System.out.println("Chef is preparing: " + order);
            Thread.sleep(2000);

            System.out.println("Chef delivered: " + order);

            orderReady = false;
            notify(); // notify waiter

        } catch (InterruptedException e) {
            System.out.println(e);
        }
    }
}

// Waiter Thread
class Waiter extends Thread {
    Kitchen kitchen;

    Waiter(Kitchen kitchen) {
        this.kitchen = kitchen;
    }

    public void run() {
        String[] orders = {"Pizza", "Burger", "Pasta"};

        for (String o : orders) {
            kitchen.placeOrder(o);
            try {
                Thread.sleep(1000);
            } catch (Exception e) {}
        }
    }
}

// Chef Thread
class Chef extends Thread {
    Kitchen kitchen;

    Chef(Kitchen kitchen) {
        this.kitchen = kitchen;
    }

    public void run() {
```

```
        for (int i = 0; i < 3; i++) {
            kitchen.prepareOrder();
        }
    }
}

public class Main {
    public static void main(String[] args) {
        Kitchen kitchen = new Kitchen();

        Waiter w = new Waiter(kitchen);
        Chef c = new Chef(kitchen);

        w.start();
        c.start();
    }
}
```

Output:

```
Waiter placed order: Pizza
Chef is preparing: Pizza
Chef delivered: Pizza
Waiter placed order: Burger
Chef is preparing: Burger
Chef delivered: Burger
Waiter placed order: Pasta
Chef is preparing: Pasta
Chef delivered: Pasta
```